```
--MATIAS COLAN
--DAYANNA VIZCARRA
revelarpalabra :: [Char] -> [Char] -> Char ->[Char]
revelarpalabra [] [] = []
revelarpalabra (x:xs) (y:ys) =
  if x == letra
  then letra : revelarpalabra xs xy letra
  else y : revelarpalabra xs xy letra

palabracompleta :: [Char] -> [Char] -> Bool
palabracompleta progreso target = progreso == target

ocultarpalabra :: Int -> [Char]
ocultarpalabra 0 = []
ocultarpalabra n = "-" ocultarpalabra (n-1)

juego :: [Char] -> [Char] -> Int -> IO()
juego progreso target vidas = do
  putStrLn $ "\npalabra" ++ progreso
  putStrLn $ "Vidas restantes:" ++ show vidas
  if vidas == 0
  then putStrLn $ "perdiste"
  else if palabracompleta progreso target
    then putStrLn $ "Ganaste"
    else do
    putStrLn $ "letra"
    (c:_) = <- getline
    let nuevo = revelarpalabra target progreso c
    if nuevo /- progreso
    then do
      putStrLn $ "letra correcta"
      juego nuevo target vida
    else do
      putStrLn $ "incorrecto"
      juego progreso target (vidas-1)

main :: IO()
main = do

seed<- getline
let s = `mod` (from Enum(last seed) -39)
let lista = ["hola", "mundo", "casa", "campo", "selva", "arboles", "mono", "viaje", "bus",
"accidente"]
let target = lista !! s
let progreso = ocultarpalabra(length target)
juego progreso target 6
```